

Final Project Requirements

CSE 340 Web Backend Development

Overview

For your final project, you will build a fully functional, server-side rendered web application that demonstrates your mastery of backend development concepts.

This is not a checklist assignment. You are being evaluated on whether you have built a complete, working application that shows genuine understanding of backend development principles.

Technology Stack

Required Technologies:

- Node.js with Express.js as the backend framework
- EJS or Liquid.js for rendering views
- ESM (ECMAScript Modules), no CommonJS (require is not allowed)
- PostgreSQL for the database
- Deployed on Render with a connected PostgreSQL database

Core Concepts You Must Demonstrate

Your project must show evidence that you understand and can implement:

1. Database Design and Relationships

- Multiple related tables with proper foreign keys
- Normalized data structure (no dumping everything into one table)
- Appropriate data types for each column
- Thoughtful design decisions about relationships between tables
- Appropriate use of CASCADE or SET NULL on foreign keys

2. User Authentication and Authorization

- Session-based authentication using express-session (no JWT)
- Password hashing (bcrypt or similar)
- Multiple user roles with different permissions:
 - **Admin/Owner:** Full control over system and content
 - **Secondary Role:** Limited permissions appropriate to your site (employee, moderator, vendor, etc.)
 - **Standard User:** Basic account with limited access
- Protected routes that check authentication and authorization
- Secure session configuration

3. Server-Side Rendering and Routing

- All pages rendered server-side using EJS or Liquid.js
- Dynamic routing for specific resources (e.g., /product/:id, /vehicle/:id)
- Proper use of layouts and partials to avoid code repetition
- Clean URL structure that makes sense
- Optional but encouraged: Query parameters for sorting, filtering, or pagination

4. MVC Architecture and Code Organization

- Clear separation of concerns: Models, Views, Controllers (or Routes if you prefer)
- Middleware for authentication, validation, and error handling
- Global error handler to manage errors in one centralized location
- Appropriate use of try/catch:
 - If you prefer, in model files where you want to return data consistently (even empty results on error) rather than throwing errors up the chain
 - Not using in places where we would want errors to go up the chain to your global error handler
- ESM import/export syntax throughout

5. Dynamic Content Management

- Admin role can manage core site content (products, listings, articles, etc.)
- Content is stored in the database and displayed dynamically
- Users can interact with content based on their role
- Changes by admins are immediately reflected on the site

6. User Interaction System

- Users can submit content tied to their account (reviews, comments, ratings, etc.)
- User-generated content is stored in the database with proper relationships
- Users can view, edit, and delete their own submissions
- Validation prevents empty or inappropriate submissions

7. Multi-Stage Workflow System

Your site must include a process that moves through multiple stages. Examples include:

- Service/repair requests (submitted, in progress, completed)
- Support ticket system (open, assigned, resolved)
- Approval workflow (submitted, under review, approved/rejected)
- Order/booking system (requested, confirmed, completed)
- Shopping cart and checkout process (cart, checkout, order placed, fulfilled)
- Contact form with responses (received, replied, closed)
- Reservation system (requested, confirmed, checked in, completed)

These are just examples. Any workflow appropriate to your application that involves multiple stages and status tracking will work. Users should be able to submit something, see its current status, and view its history.

8. Admin Dashboard

A management interface where authorized users can:

- View and manage users and their roles
- Add, edit, and delete core site content
- View and respond to user submissions (messages, tickets, requests)

- Moderate user-generated content if needed (keep this simple: delete or flag inappropriate content)
- See relevant operational data

You may also include a dashboard for standard users if you wish. This could allow them to view their submitted content, account information, order history, or other personal data relevant to your application.

9. Security and Validation

- SQL injection prevention through parameterized queries
- Input validation on all forms
- Sanitization of user inputs
- Secure session configuration
- Appropriate error messages (do not leak system details)

10. Deployment

- Functioning deployment on Render
- PostgreSQL database properly connected in production
- Environment variables configured correctly
- No development-only code running in production

Required Documentation and History

You must include a **README.md** at the root of your repository that contains:

1. **Project Description:** What your site does and who it is for
2. **Database Schema:** An image of your entity relation diagram (ERD) exported from pgAdmin showing your tables and relationships
3. **User Roles:** Explanation of each role and what they can do
4. **Test Account Credentials:** Username or email for one account of each role type, do not include the password in the README, but use **P@\$\$w0rd!** for all users
5. **Known Limitations:** Any features you did not complete or bugs you are aware of

Additionally, your GitHub repository must show:

- Minimum 15 substantial commits (not simple one-line fixes)

- Organized folder structure
- Clean, readable code with consistent formatting

Choose Your Path

You have two options for your final project:

Option A: Design Your Own Project

Come up with your own idea that incorporates all the core concepts above. Your project should be a complete, realistic web application that serves a clear purpose.

Examples that work well:

- Rental/booking platform (equipment, venues, appointments)
- Community site (forum, recipe sharing, local events)
- Service business site (salon, tutoring, consulting)
- Marketplace (buy/sell, classifieds, auctions)
- Educational platform (courses, tutorials, resources)
- E-commerce site with shopping cart
- Event management system
- Task/project management tool

Before you build, make sure your idea includes clear opportunities for:

- Multiple user roles with different permissions
- Content that admins manage
- User interactions and submissions
- A multi-stage process or workflow

Option B: Build a Used Car Dealership

If you prefer clear requirements, build a used car dealership website with these features:

Public Pages:

- Home page with featured vehicles
- Browse vehicles by category (Trucks, Vans, Cars, SUVs)

- Individual vehicle detail pages with images, specs, and price
- Contact form (saves to database)

User Features (must be logged in):

- Leave reviews on vehicles
- Edit/delete own reviews
- Submit service requests for their vehicle (oil change, inspection, etc.)
- View history of service requests and their status

Employee Dashboard:

- Edit vehicle details (price, description, availability)
- Moderate/delete inappropriate reviews
- View and manage service requests
- Update service request status (Submitted, In Progress, Completed)
- Add notes to service requests
- View contact form submissions

Owner Dashboard (Full Admin):

- Everything employees can do, plus:
- Add, edit, and delete vehicle categories
- Add, edit, and delete vehicles from inventory
- Manage employee accounts (optional, can be hardcoded)
- View all system activity and user data

Database Requirements:

- Users table (with role field)
- Vehicles table
- Categories table (linked to vehicles)
- Reviews table (linked to users and vehicles)
- Service requests table (linked to users, with status tracking)

- Contact messages table
- Vehicle images table (one-to-many with vehicles)

What Success Looks Like

You will know you are on the right track if:

- Someone could actually use your website for its intended purpose
- Your database structure would make sense to another developer
- Your code is organized well enough that you could hand it off to someone else
- The site works reliably without breaking
- Different user roles have genuinely different experiences
- Your deployment is stable and uses production-ready practices

What Will Result in a Failing Grade

- Incomplete or non-functional authentication system
- Major security vulnerabilities (plain text passwords, SQL injection risks, no input validation)
- Poorly designed database (single table, no relationships, inappropriate data types)
- Broken deployment or inability to connect to the database
- Missing core concepts listed above

Submission Requirements

Before the deadline, you must submit:

1. GitHub repository URL (with meaningful commit history)
2. Live deployment URL on Render

Your deployment must:

- Be accessible and functioning
- Have a PostgreSQL database properly connected
- Include one test account for each user role (credentials in README)

Final Thoughts

This project is your opportunity to demonstrate that you can build real, functional web applications. Focus on understanding what you are building and why. Build something you are proud of, something that works, and something that demonstrates genuine mastery of backend development.

Do not aim for the bare minimum. Be creative, experiment with ideas, and challenge yourself to implement features that interest you. Practice your skills and have fun with this project. If you approach it with curiosity and a desire to learn rather than just checking boxes, you will not only pass easily but also build something meaningful that showcases your abilities. The best projects come from students who genuinely engage with the material and enjoy the process of building.